A Multi-Dimensional, Per-Pass Empirical Study of the LLVM Optimization Pipeline

Federico Bruzzone and Walter Cazzola

***Abstract*—Quantifying the marginal impact of individual optimization passes underpins phase ordering, pass selection, optimization design, and analysis of pass/hardware interactions. In LLVM, the standard backend for C/C++, Rust, and ML stacks via MLIR, interactions among passes, measurement noise, and pipeline scale make this difficult. We present a systematic empirical study of the LLVM `-O3` pipeline. We decompose it into 113 cumulative per-pass prefixes and measure execution time, compile time, binary size, hardware counters, and RAPL energy on 30 PolyBench/C kernels (84,750 measurements). Transitions whose binary is byte-identical to their predecessor give us a built-in null distribution, which we use to calibrate every significance claim. On these compute-bound affine kernels, the pipeline is non-monotone: 36% of the transitions that significantly change runtime are regressions, found in 28 of 30 kernels. Gains are back-loaded (the median kernel needs 84.8% of the pipeline for 80% of its speedup) and concentrated in a small core of passes, a result that holds under both absolute and scale-free impact measures. IR instruction count is an unreliable runtime predictor, and runtime-targeted passes are *de facto* energy-targeted (30–60% savings). Finally, we bound the speedup lost to non-monotone behavior from above by 46.35% and decompose it: 9.7 points are measurement noise, 15.3 points are transient dips repaired within three passes, and only 21.3 points persist to the end of the pipeline. These findings inform pass pruning, cost-model calibration, and autotuning.**

***Index Terms*—Compiler optimizations, Optimizing Compilers, LLVM, Multi-dimensional Empirical study**

I. Introduction

SINCE the dawn of computing, compiler optimizations have bridged high-level abstractions and efficient machine code [1]–[6]. Optimizing compilers [7], [8] use analysis passes to guide transformations [9]–[11] that enhance performance without altering observable behavior [12]–[14], and modern architectures make this a delicate interplay of interdependent passes [9]. LLVM [15] exemplifies this complexity. It is the standard backend for C/C++, Rust [16], Julia [17], and, via MLIR [18], for *machine learning* (ML) and *deep learning* (DL) stacks [19], [20] built on ML/DL optimizing compilers [21]–[24] such as OpenXLA[1] and TVM [25].

The Phase-Ordering Problem. The optimal sequence of optimizations is program-specific [26]–[28]. The *phase-ordering problem* [29]–[31], i.e., finding the optimal pass order, is undecidable in general [32]–[34], and optimization selection is itself hard [35]–[37]. *Compiler autotuning* [38]–[44] searches this large space [45]–[47], but its effectiveness depends on accurately estimating pass impact, which is complicated by inter-pass dependencies, noisy measurements, and scale [48]. Some transformations even degrade performance [49], [50]. Despite this, both industry and academia still rely on predefined, *one-size-fits-all* pipelines (e.g., `-O3`) for efficiency and because users have limited knowledge of the optimization process [51]–[53].

Motivation and Gap. Quantifying the impact of individual passes matters to compiler engineers and end users [54]–[56]. It enables (i) better phase ordering, (ii) better pass-selection heuristics, (iii) informed optimization design, and (iv) analysis of pass/hardware interactions [33]. Yet the folklore about `-O3` ("most passes do little", "passes sometimes undo each other") has rarely been measured on a production pipeline. The closest studies measure execution time only, either on a curated subset of LLVM passes [57] or on GCC/ICC [58]. They therefore cannot see trade-offs across objectives: a pass that improves runtime may increase binary size or degrade cache locality. Phase-ordering work, in turn, quantifies loss only relative to an alternative order found by search [49], [59], which conflates the cost of the default order with the reach of the search. Finally, pass-level studies rarely calibrate what counts as a "real" per-pass effect against measurement noise, so small deltas are easily over-interpreted.

Our Study. We address these gaps with a systematic empirical study of the full LLVM `-O3` pipeline (113 pass positions, not a curated subset), quantifying the effect of each pass on runtime, compile time, binary size, nine hardware counters, and RAPL energy[2]. We formulate the following research questions (RQs):

> **RQ$_1$.** *What is the marginal impact of individual optimization passes within the LLVM `-O3` pipeline?*

> **RQ$_2$.** *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

> **RQ$_3$.** *How do optimization passes affect hardware-level behavior as reflected by performance counters?*

> **RQ$_4$.** *How much speedup does non-monotone pipeline behavior forfeit, how much of it persists to the end of the pipeline, and which passes are responsible?*

To answer them, we built an infrastructure that decomposes an optimization pipeline into its constituent passes and builds a sequence of cumulative prefixes, each extending the previous

F. Bruzzone and W. Cazzola are with the Computer Science Department, Università degli Studi di Milano, Milan, Italy (e-mail: federico.bruzzone@unimi.it; cazzola@di.unimi.it).

[1] openxla.org

[2] Running average power limit [60]–[62].

one by a single pass. For every prefix, we measure the newly added pass relative to both the preceding prefix and the `-O0` baseline, enabling both a *differential* per-pass analysis and a *cumulative* analysis of the whole trajectory. Because many passes leave the binary untouched on a given kernel, the study contains its own null distribution: we use the 2,434 transitions with byte-identical binaries to calibrate every significance threshold we apply.

Contributions. The paper contributions are:

- the first per-pass characterization of a complete production `-O3` pipeline across five dimensions (runtime, compile time, binary size, hardware counters, energy), including, to our knowledge, the first per-pass energy profile of LLVM;
- a noise-calibrated analysis of pass impact, showing that non-monotonicity, the concentration of gains in a small core of passes, and their back-loading are robust to estimator choice and to absolute versus scale-free measures, and that IR instruction count is an unreliable runtime proxy;
- a search-free upper bound L on the speedup forfeited by non-monotone behavior, decomposed per benchmark into measurement noise, transient dips, and persistent loss, and attributed to individual passes; and
- a reusable, configuration-driven infrastructure for pipeline decomposition and multi-metric measurement, released with all raw data.

The rest of the paper is organized as follows. §II provides background, §III describes the methodology, §IV presents the results, §V discusses their implications, §VI addresses threats to validity, §VII reviews related work, and §VIII concludes.

II. The LLVM Optimization Pipeline

LLVM is a modular SSA-based compilation framework [63]–[65]. Its `opt` tool applies sequences of analysis and transformation passes that constitute the optimization pipeline. The *new* pass manager[3] organizes the pipeline hierarchically through nested pass managers operating at different IR granularities: **`module`**, **`function`**, **`cgscc`**, **`loop`**, and **`loop-mssa`**. Passes may therefore appear multiple times within the pipeline at different levels of the hierarchy. At the top level, the **`module`** manager coordinates module-wide optimizations (e.g., `called-value-propagation` [66], [67]) and invokes nested managers. The remaining managers apply passes that operate respectively on (i) individual functions (e.g., `mem2reg` which promotes stack allocations to virtual registers [68], [69], and `sroa` which performs *scalar replacement of aggregates* [70], [71]), (ii) strongly connected components of the call graph (e.g., interprocedural `inline` [72]), (iii) loops (e.g., `loop-unroll` [73], [74]), and (iv) loops in memory SSA form (e.g., `licm` which performs *loop-invariant code motion* [75], [76]). Passes execute sequentially over an evolving IR. Consequently, the effect of a pass depends on the transformations that precede it, making optimization pipelines inherently *order-dependent*, *non-commutative*, and *non-additive*.

[3] `llvm.org/docs/NewPassManager.html`

III. Experimental Methodology

We develop an infrastructure that decomposes LLVM pipelines into passes and measures their marginal impacts. The four RQs introduced in §I are operationalized as follows.

RQ$_1$. We quantify the marginal impact of each optimization pass by measuring the delta of its performance relative to both the preceding pipeline state and the `-O0` baseline.

RQ$_2$. We quantify pass-level trade-offs across execution time, compile time, and binary size.

RQ$_3$. We characterize the effect of optimization passes on hardware behavior by tracking the evolution of performance counters across the pipeline.

RQ$_4$. We bound the speedup forfeited by non-monotone behavior with a search-free metric L, decompose it into noise, transient, and persistent components, and attribute it to pass positions.

Evaluation Infrastructure. An *orchestrator* coordinates pipeline construction, execution, and metric collection. Benchmarks, metrics, and experimental parameters are specified in TOML, allowing new studies without modifying the core implementation. Although pipeline-agnostic, in this study the infrastructure targets LLVM's `-O3` pipeline. It first extracts and flattens the pipeline into an ordered sequence of passes $p_0, p_1, \ldots, p_n$, then constructs cumulative prefixes $P_i = p_0 \rightarrow \cdots \rightarrow p_i$, preserving the original execution order and pass dependencies. The design is similar to that of Chen *et al.* [58], who measured execution time only on GCC/ICC. We extend it to the complete LLVM pipeline and to five dimensions. Each prefix is applied independently to the same unoptimized IR I_0, producing an optimized IR I_i and executable binary B_i. Comparing consecutive binaries B_i and B_{i-1} yields the marginal impact of pass p_i within its execution context. We chose cumulative prefixes over leave-one-out ablation because they measure each pass exactly as `-O3` deploys it, after every pass that precedes it, and because the resulting trajectory is what RQ$_4$ requires. Leave-one-out answers a different, counterfactual question (what if p_i were deleted?), which mixes the effect of p_i with the reaction of all later passes to its absence. Each flattened entry keeps its pass-manager nesting (e.g., `function(. . . )`), so a function pass in prefix P_i runs on every function, exactly as in `-O3`. Analyses are computed on demand by the analysis manager and are not enumerated as separate entries.

Benchmark Selection. We use the 30 kernels of PolyBench/C 4.2.1[4] in default mode. To keep the 95% confidence interval below 1% of the mean, benchmarks must be self-contained (single translation unit, `libm` only), deterministic (single-threaded), and execution-time tractable. PolyBench satisfies these requirements, whereas larger suites (e.g., SPEC or MultiSource) would either mask single-pass effects or exceed our repetition budget. Despite its modest size, PolyBench covers a heterogeneous set of workloads [77]. Its six categories span the computation patterns most targeted by `-O3`, including loop nests, affine accesses, reductions, stencils, and triangular solvers. PolyBench is also the *de facto* benchmark suite for autotuning and ML-based pass-selection studies [40],

[4] `github.com/MatthiasJReisinger/PolyBenchC-4.2.1/polybench.pdf`

facilitating comparison with prior work. We use the default dataset configuration, which maximizes level-1 cache (L1) utilization while keeping working sets within the last-level cache (LLC), thereby exposing optimization effects without being dominated by memory bottlenecks. PolyBench is also relevant to the MLIR-based argument of §I. Modern ML stacks lower tensor operations to LLVM IR through MLIR, where code generation is ultimately driven by the same `-O3` pipeline studied here. Many MLIR-generated kernels are structurally similar to PolyBench kernels: matrix multiplications and convolutions map naturally to `gemm`, `2mm`, and `3mm`; attention and reduction patterns resemble `atax`, `bicg`, `gesummv`, and `mvt`; normalization workloads resemble `correlation` and `covariance`; and iterative PDE-style computations resemble `jacobi-1d`, `jacobi-2d`, `heat-3d`, `seidel-2d`, and `fdtd-2d`. These structural correspondences suggest that our findings may transfer to MLIR-based ML pipelines, although direct validation remains future work.

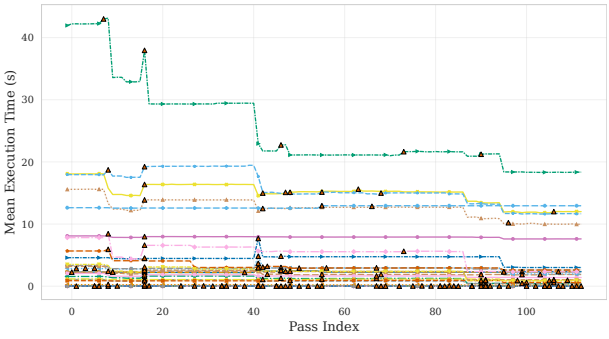
Metric Collection. For each pipeline variant, we measure execution time, binary size, and end-to-end compilation time (wall-clock time for `clang+opt+llc`, excluding linking). Hardware behavior is characterized using `perf stat` and nine validated events [78]: cycles and retired instructions, whose ratio gives *instructions per cycle* (IPC, a throughput measure) and its inverse CPI; L1 data-cache (D1), L1 instruction-cache (L1-I), and last-level cache (LLC) misses, which expose data and code locality; branch misses, which expose control-flow predictability; and package energy via RAPL (`power/energy-pkg/`, DRAM/GPU excluded). Runtime is roughly cycles divided by frequency, so a pass can speed up a kernel either by removing instructions or by raising IPC; the counters tell the two apart. Cache references, peak RSS, page faults, and context switches are collected as confound checks.

Experimental Setup. All experiments ran on Intel Core i9-12900KF (Alder Lake, 8P+8E cores, 5.2 GHz max), 128 GB RAM, Fedora Linux 43 (kernel 6.19), with cache hierarchy: 640 KiB D1, 768 KiB L1-I, 14 MiB L2, 30 MiB L3. We used LLVM 21.1.8 (release, no assertions, no PGO). The `-O3` pass sequence was extracted via `opt -print-pipeline-passes -O3` and flattened. The pipeline has 113 distinct invocations (including repeats of `instcombine`, `simplifycfg`, `early-cse`). Our protocol entails 84,750 measurements (50,850 runtime, 33,900 hardware-counter runs). Runtime measurements use `hyperfine` (3 warmup, 15 timed runs), while hardware counters are collected with `perf stat` (2 warmup, 10 timed runs) in a separate batch. Ten runs are sufficient to keep the 95% confidence interval below 1% of the mean for all counter metrics.

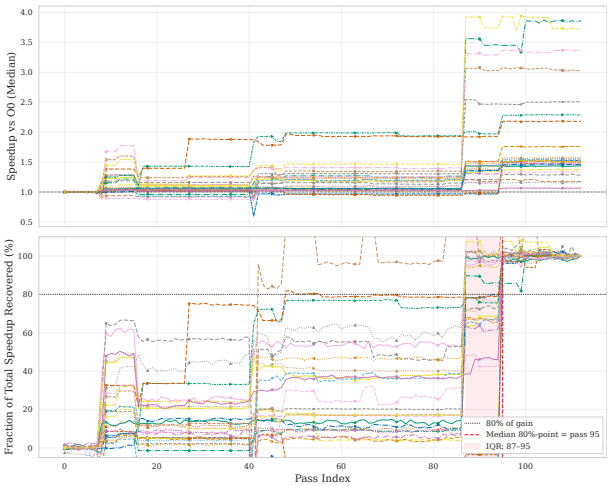
Noise Mitigation. To reduce measurement variability, benchmarks are pinned to a dedicated P-core using `taskset`, executed under real-time FIFO scheduling (`chrt`), run with the CPU governor set to `performance` (`cpupower`), and evaluated with address-space layout randomization (ASLR) disabled (`setarch -R`). Hardware performance counters are collected in separate runs to avoid instrumentation overhead.

Analysis Protocol and Null Calibration. The 30 kernels and 113 prefixes yield 3,390 *transitions* (`-O0` to prefix 0, and each prefix to the next). For transition i we use the scale-free relative delta $r_i = (T_i - T_{i-1})/T_{i-1}$ on median runtime, so $r_i > 0$ is a slowdown. A transition is a *significant* change when $|r_i| > 1\%$ and a two-sided Mann–Whitney U test on the 15 timed runs gives $p < 0.05$. Benjamini–Hochberg correction across all transitions changes the counts by at most one. We calibrate this criterion on a natural null: 2,434 transitions (71.8%) produce a binary byte-identical to the previous one, so any measured difference is noise. On these, $p < 0.05$ alone fires in 25.7% of cases, reflecting slow drift between measurement batches [79], while our combined criterion fires in only 22 cases (0.9%). We therefore never rely on p-values or effect sizes alone, and report 0.9% as the empirical false-positive rate of every count below.

Artifacts, including the enumerated pass sequences, are available on GitHub[5] and Zenodo[6].



(a) runtime evolution with 95% confidence interval.



(b) cumulative speedup S_i (*top*) and normalized gain $G_b(i)$ (*bottom*).

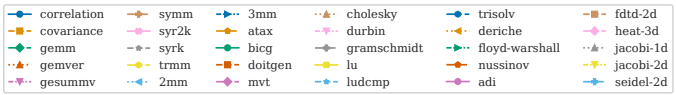
Fig. 1: Runtime evolution and optimization saturation.

IV. Results

We report median statistics as recommended in [79], [80], and mean statistics with a 95% confidence interval. All multi-benchmark figures use the following legend (unique color,

[5] `github.com/FedericoBruzzone/llvm-passview`
[6] `https://doi.org/10.5281/zenodo.20799028`

marker, line style per kernel):



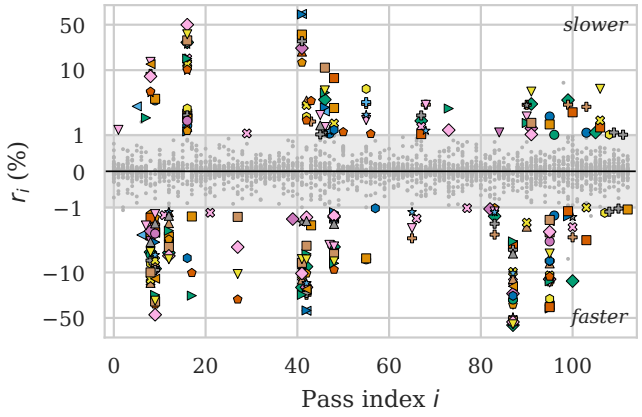
A. RQ_1: Marginal Performance Impact

Fig. 1a plots, for each kernel, median execution time (y) against the prefix index $i \in [0, 112]$ (x), with $i = -1$ denoting `-O0`; shaded bands are 95% confidence intervals, which stay below 1% of the mean throughout. Fig. 1b shows the same trajectories as speedup over `-O0`, S_b^i (top), and as normalized gain $G_b^i = (S_b^i - 1)/(S_b^* - 1) \in [0, 1]$ (bottom), the fraction of the final speedup S_b^* reached at prefix i.

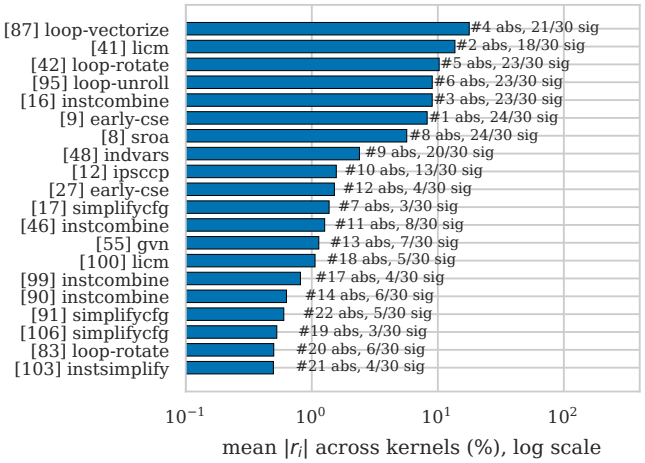
Non-monotonicity. Of the 3,390 transitions, 291 significantly change runtime (§III), and 106 of these (36%) are *regressions*. Restricting to the 956 transitions that change the binary leaves 97 regressions and 172 improvements, again 36%. Regressions occur in 28 of 30 kernels, have median magnitude $+2.7\%$ (maximum $+73\%$), and are far above the 0.9% false-positive rate of the null. Three passes cause 68% of them: `instcombine` (39), `simplifycfg` (16), and `licm` (11). Counting every nominal increase ($r_i > 0$) instead would label 48% of all transitions as regressions, most of them noise.

Back-loading. Across the 27 kernels whose final speedup exceeds 1, the median prefix needed to reach 80% of the final speedup contains 84.8% of the passes (IQR 77.7%–84.8%). Removing the last $\sim$15% of the pipeline thus sacrifices roughly 20% of the attainable speedup, contradicting the intuition that most gains occur early. The G_b curves have an S-shaped profile: an initial canonicalization phase, a steep growth region dominated by loop transformations and vectorization, and a final plateau. At $\sim$25% of the pipeline almost all kernels remain at $S \approx 1$; by $\sim$50%, 70% have crossed $1.2\times$; at the end, 80% reach $1.5\times$ or more. Several curves temporarily exceed $G_b = 1$: intermediate states that outperform the final binary, which we quantify in §IV-B and §IV-D. Gains concentrate on dense-matrix and stencil kernels (`gemm` $3.86\times$, `jacobi-2d` $3.72\times$, `heat-3d` $3.37\times$, `fdtd-2d` $3.02\times$), while `correlation` ($0.96\times$), `covariance`, and `seidel-2d` ($0.98\times$) finish slower than `-O0`.

Concentration. Fig. 2a plots r_i for every transition: grey dots are not significant (the shaded band is $\pm 1\%$), colored markers are significant changes of the kernel given in the legend, and slowdowns lie above zero. Most transitions are inert. Only 32.6% modify the IR and 28.2% the binary; 58 of the 112 pass positions never modify the IR of any kernel, and only 21 modify all 30. Significant changes cluster in three bursts: early canonicalization (passes 8–18), loop canonicalization (41–48), and vectorization/unrolling (87–95). Fig. 2b ranks pass positions by mean $|r_i|$ across kernels. Being relative, this measure weights every kernel equally regardless of its runtime scale. It agrees with the absolute-ms ranking (Spearman $\rho = 0.79$; 9 of the top-10 and 18 of the top-20 positions coincide), so concentration is not an artifact of long-running kernels. Under the relative measure, 14 positions (12.4%) account for 80% of the total impact, and within each kernel a median of 10.6% of the passes (IQR 5.5%–15.9%) account for 80% of $\sum_i |\log(T_i/T_{i-1})|$. The core is `loop-vectorize`, `licm`, `loop-rotate`, `loop-unroll`, `instcombine`, `early-cse`, and `sroa`. The two measures differ only in emphasis: `early-cse` ranks first in milliseconds because it fires on long-running kernels, whereas `loop-vectorize` ranks first in relative terms. The column of significant kernels separates *universal* passes (e.g., `early-cse`, `sroa`, 24/30) from *program-specific* ones (e.g., `gvn`, 7/30). The ranking is *per occurrence*: passes scheduled several times (`instcombine`, `simplifycfg`, `licm`, `early-cse`) receive one entry per position, and only the first occurrence of each is in the core. We do not report Cohen's d: near-deterministic transitions have vanishing variance, which inflates $|d|$ without bound, and the null shows that significance alone is unreliable. Across categories, `blas` and `stencils` see large late-pipeline gains (medians above $1.5\times$ and $2.0\times$), `datamining` stays near $1.0\times$, and `solvers` and `medley` improve near-monotonically.



(a) relative per-pass delta r_i for every transition (symmetric-log scale).



(b) top-20 pass positions by mean $|r_i|$, with rank under the absolute (ms) measure and number of kernels where the transition is significant.

Fig. 2: Scale-free marginal impact of each pass.

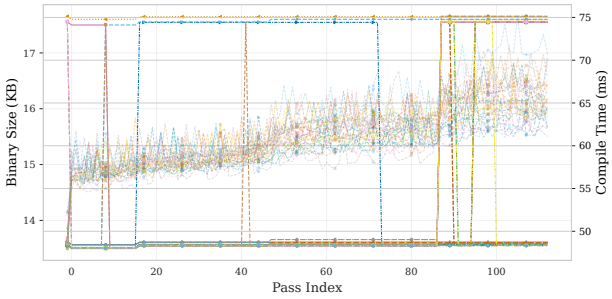


Fig. 3: Compile time and binary size trajectories.

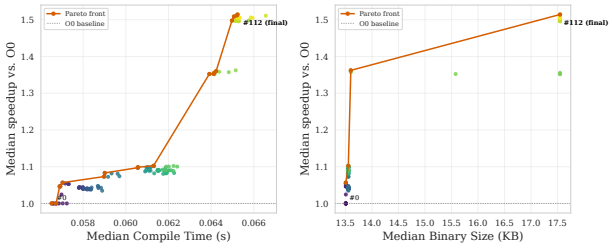


Fig. 4: Pareto views of the multi-objective trade-off, each point aggregating all benchmarks at one variant index.

Finding 1 (RQ_1)

The `-O3` pipeline is *non-monotone*: 36% of the transitions that significantly change runtime are regressions, present in 28 of 30 kernels and caused mostly by `instcombine`, `simplifycfg`, and `licm`. Impact is concentrated: about 12% of the pass positions account for 80% of it, under both absolute and scale-free measures, while two thirds of the transitions do not even modify the IR. The realized gain is *back-loaded*: of the 27 kernels that speed up, 24 cross G_b=0.8 right after `loop-vectorize` (pass 87, 10 kernels) or `loop-unroll` (pass 95, 14 kernels), placing the median crossing at 84.8% of the pipeline.

B. RQ_2: Multi-Dimensional Trade-offs

RQ_2 asks whether runtime improvements are bought at the price of compile-time or footprint regressions. In Fig. 3, the bold step lines (left axis) are per-kernel binary sizes and the thin faded lines (right axis) are per-kernel compile times. Compile time grows near-monotonically, accelerating where loop and inlining passes fire. Binary size is *non-monotone*: early passes shrink it, while late vectorization and unrolling inflate it in a few discrete steps. Fig. 4 aggregates all kernels per prefix (median speedup against median compile time, left, and median binary size, right; the orange line is the Pareto front). The compile-time frontier is L-shaped: most speedup arrives at the pipeline tail (∼prefix 80) for modest extra compile cost. The size frontier is two-step: an initial wave of speedups is free in size, while the final jumps coincide with abrupt size increases.

Is the final `-O3` *binary Pareto-optimal?* Per kernel, in 29 of 30 cases some earlier prefix is nominally faster than the final binary at no larger size. This dominance is real but thin. The best such prefix is significantly faster in 24 kernels, but only

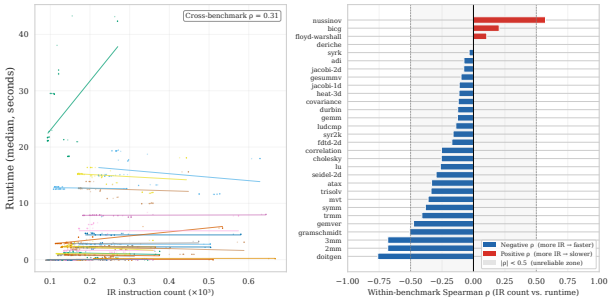


Fig. 5: *Left:* IR instruction vs. median runtime by benchmark; cross-benchmark Spearman $\rho = 0.31$. *Right:* within-benchmark Spearman ρ for all 30 benchmarks.

by a median 0.8% (more than 1% in 10 kernels), and it usually lies in the last 15 passes (median position 103), after binary size has reached its final value. Only four kernels have a prefix that is strictly smaller *and* faster: `correlation` (+6.5% speed), `covariance` (+5.1%), `seidel-2d` (+3.1%), and `symm` (+2.5%). They are the kernels with the largest persistent loss in §IV-D.

IR size as a runtime proxy. Fig. 5 shows that static IR-instruction count cannot serve as a proxy for runtime. The left panel (IR count against median runtime, one fitted trend per kernel) yields a weak cross-kernel Spearman $\rho = 0.31$. The right panel gives the within-kernel ρ: 27 of 30 kernels have *negative* ρ (more IR, faster code), because the passes that expand the IR (unrolling, vectorization) are those that reduce runtime. Only `nussinov`, `bicg`, and `floyd-warshall` are positive, and most bars fall within the $|\rho| < 0.5$ zone regardless of sign.

Finding 2 (RQ_2)

Compile time grows near-monotonically, and binary size is non-monotone: early passes shrink it, late vectorization and unrolling inflate it. The final `-O3` binary is nominally Pareto-dominated on (size, speedup) in 29 of 30 kernels, but the margin is significant in 24 and exceeds 1% in only 10. A strictly smaller and faster prefix exists only for `correlation`, `covariance`, `seidel-2d`, and `symm` (2.5–6.5% faster). The `opt` stage accounts for less than half of total compile time, and its share is essentially kernel-independent, so pass-budget decisions can be made per pipeline, not per program. Static IR-instruction count is *not* a reliable proxy for runtime under pipeline-prefix variation.

C. RQ_3: Hardware-Level Insights

RQ_3 asks how IR-level transformations propagate to hardware behavior. Fig. 6 has one panel per kernel; rows are the ten pass positions with the largest absolute impact (nine of which are also in the relative top-10 of Fig. 2b) and columns are counters. Each cell is the percentage change relative to `-O0`, blue for a decrease and red for an increase, with the colormap anchored at the 95th-percentile magnitude so that outliers do not flatten the remaining cells. The top-10 positions form three groups. Scalar canonicalization (8 `sroa`,

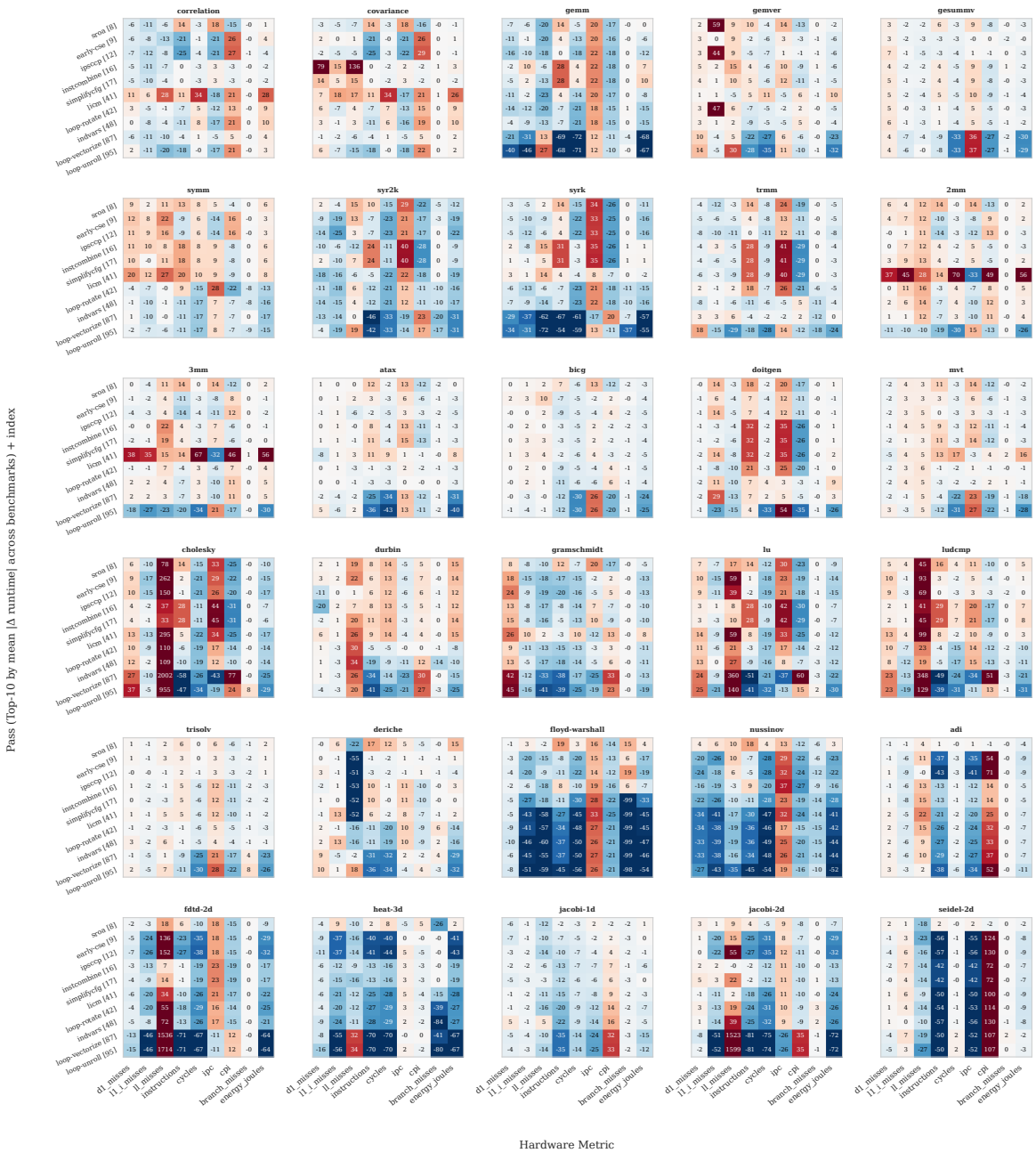


Fig. 6: Per-benchmark 5×6 grid of hardware-counter signatures at the top-10 most impactful pass positions.

9 `early-cse`, 16 `instcombine`, 17 `simplifycfg`) gives the most uniform IPC gain (median +11–14%, 26–28/30 kernels) at the cost of transient instruction inflation (+5–13%), while cycles still fall by 3–4%. Propagation (12 `ipsccp`) consistently reduces instructions (6–7%), cycles (5%), and energy (4–6%) in 19–28 kernels. Loop transformations and vectorization (41 `licm`, 42 `loop-rotate`, 48 `indvars`, 87 `loop-vectorize`, 95 `loop-unroll`) span the widest range. Pass 41 is the most variable (cycles −47% to +70%), while passes 42 and 48 are uniformly beneficial (instructions −7 to −11%, energy −8 to −9%, 25–28 kernels). Pass 95 gives the deepest suite-wide reductions (median instructions −37% in 29/30 kernels, cycles −33%, energy −29%) but splits the LLC column: deep blue for most kernels, deep red for the `lu`/`ludcmp`/`cholesky` triad, where aggressive unrolling of dense triangular loops expands the data working set. The same pass thus produces qualitatively different microarchitectural signatures across kernels.

Fig. 7 follows six metrics along the pipeline: each colored line is a kernel, and the grey line and band are the suite-wide median and IQR. In the cache and energy panels the cross-kernel spread dwarfs the median (the `-O0` IQR of D1 misses spans $0.89–48.9\times10^6$), so the median hugs the floor while the solvers and `floyd-warshall` float far above. Each median advances as a plateau broken only at the recurring hot passes (8–9, 16, 41–42, 87, 95, 106). D1 misses are nearly flat (+2.6% end-to-end). L1-I misses fall by 36.8% in two drops, at passes 9 and 87. LLC misses rise by 35.6%, with the main step at pass 87. IPC rises to its peak at pass 8

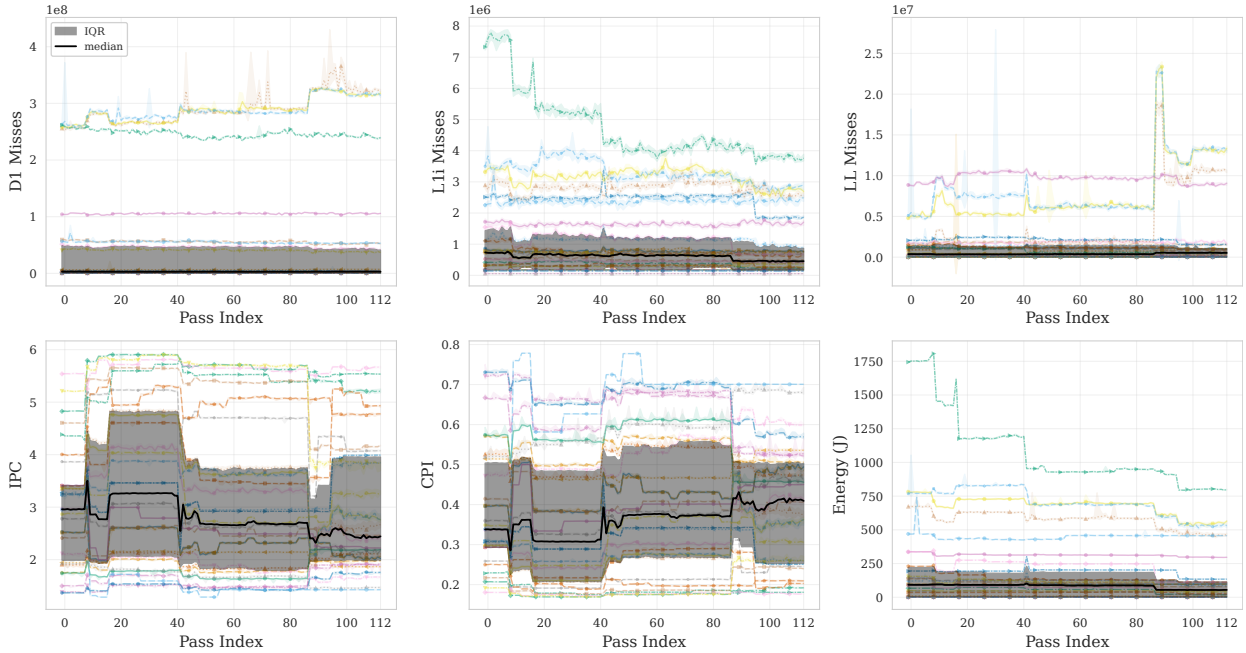


Fig. 7: Per-pass evolution of D1, LLC, and L1-I miss rates, IPC, CPI, and energy.

($2.96 \rightarrow 3.50$), collapses at pass 41, and ends at 2.44, a 17.4% *decrease* that coexists with large speedups because SIMD code executes far fewer, longer-latency instructions. Energy (RAPL package) falls by 40.9% ($92.2 \rightarrow 54.5$ J), with the dominant step at pass 87 (-35%). To our knowledge this is the first *per-pass* energy profile of the LLVM pipeline. The binary growth of §IV-B therefore does *not* cause instruction-cache pressure: L1-I misses fall with the instruction count, and its only microarchitectural cost is on the data side, where late unrolling inflates LLC misses on the dense solvers.

Finding 3 (RQ_3)

Runtime gains come from large reductions in instruction count and cycles, *not* from higher throughput: IPC *falls* (-17.4%) as vectorization trades many cheap instructions for fewer, longer-latency SIMD operations, while the D1 miss rate stays essentially flat. Energy savings track runtime savings (30–60% suite-wide median): runtime-targeted passes are *de facto* energy-targeted for compute-bound kernels. Conversely, `loop-vectorize` and `loop-unroll` *lower* L1-I misses (fewer instructions fetched); the binary-bloat trade-off of §IV-B surfaces instead as *data* working-set expansion, inflating LLC misses on the dense solvers (`lu`/`ludcmp`/`cholesky`). Branch-misprediction behavior is essentially flat: control-flow optimization is not on the critical path for PolyBench.

D. RQ_4: Non-Monotonicity Loss

The regressions of RQ_1 raise the question of how much speedup the pipeline forfeits by moving backwards. With $\Delta S_i = S_i - S_{i-1}$ the per-pass change in speedup over `-O0`,

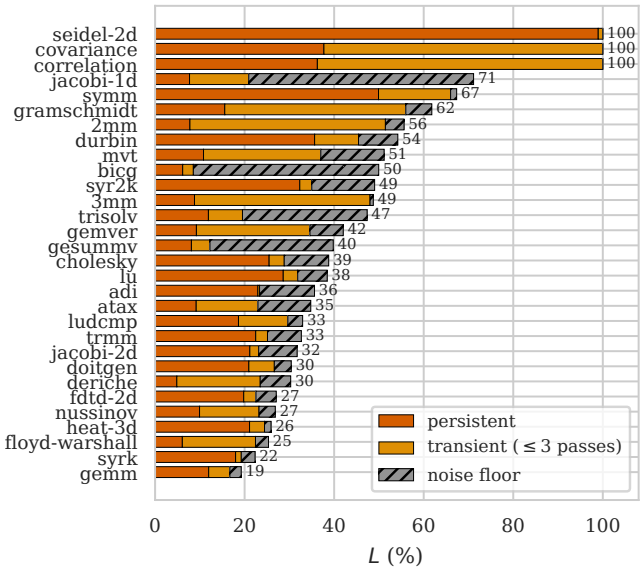


Fig. 8: Non-monotonicity loss L per kernel (bar end), split into persistent loss, transient dips repaired within three passes, and the share due to transitions with byte-identical binaries (noise floor).

we define the additive ceiling and the loss

$$S_{\text{opt}} = \sum_i \max(\Delta S_i, 0), \quad L = \frac{S_{\text{opt}} - [S_{\text{actual}}]_0^{S_{\text{opt}}}}{S_{\text{opt}}},$$

where $S_{\text{actual}} = S_{\text{final}} - 1 = \sum_i \Delta S_i$ and $[x]_0^{S_{\text{opt}}} = \min(\max(x, 0), S_{\text{opt}})$. S_{opt} is the speedup a pipeline would reach if it kept every gain and suffered no regression, so whenever the final speedup is positive L is simply the total

regression mass divided by the total gain mass: a measure of *non-monotonicity*. L is not a causal estimate of harm. Passes are non-additive (§II), so S_{opt} is generally unattainable, and a dip may be the price of a later gain. We therefore treat L as an upper bound and decompose it to separate those cases. Across kernels, mean L is 46.35% (median 39.3%, IQR 30.7%–53.5%).

Decomposition. Fig. 8 splits each kernel's L into three parts. The *noise floor* is the part that disappears when transitions with byte-identical binaries are set to $\Delta S_i = 0$: on average 9.7 points, up to 50 on `jacobi-1d`. Of the remaining regression mass, a dip is *transient* to the extent that it is recovered by the gains of the next three passes, and *persistent* otherwise. Transient dips account for 15.3 points on average (41.9% of the noise-free loss, 38.6% when averaged per kernel). The canonical example is `correlation`, where the drop of pass 41 (`licm`, $\Delta S{=}-0.26$) is 71% undone by pass 42 (`loop-rotate`, $\Delta S{=}+0.18$). Persistent loss averages 21.3 points (median 18.3) and is the component that a pipeline change could, at best, recover. The four kernels where it is largest, `seidel-2d`, `symm`, `covariance`, and `correlation`, are exactly those for which RQ_2 found a strictly smaller and faster prefix, so a part of their loss is realizable by stopping early.

Attribution. Persistent loss is concentrated: the first `instcombine` (pass 16, 23 kernels) contributes 29.9% of it, `gvn` (pass 55) 18.5%, mostly from `seidel-2d`, and the first `licm` (pass 41) 8.8%. By pass type, `instcombine`, `gvn`, `licm`, and `simplifycfg` account for 78%. The passes that deliver most of the speedup are thus also the main sources of the loss that survives to the end of the pipeline. Unlike prior phase-ordering work, which measures loss relative to an alternative ordering discovered by search [32], [49], [59], [81], L is search-free and attributed to single pass positions of the default `-O3` pipeline.

Finding 4 (RQ_4)

The additive upper bound on speedup forfeited by non-monotone behavior is $L{=}46.35\%$ on average (median 39.3%), but most of it is not recoverable loss: 9.7 points are measurement noise and 15.3 points are transient dips repaired within three passes. Persistent loss averages 21.3 points, is largest on four kernels (`seidel-2d`, `symm`, `covariance`, `correlation`), and is caused mainly by `instcombine`, `gvn`, and `licm`, the same canonicalization core that drives the gains.

V. Discussion & Implications

Pipeline Surgery. The data suggest four concrete uses, each with its limits.

1) *Truncating `-O3` is not a general win.* The faster prefixes of RQ_2 sit in the last 15 passes and gain under 1% in 20 of 30 kernels. Only for the four kernels with large persistent loss does a per-program "stop early" knob pay off (2.5–6.5%), which makes it a candidate autotuning dimension, not a default.
2) *Phase-ordering search can be focused.* Persistent loss is concentrated in three positions (the first `instcombine`, `gvn`, the first `licm`). A repair search that perturbs only these positions, or guards them with "do not run p_i on IR matching signature X", explores a space orders of magnitude smaller than full reordering.
3) *Inert passes are a free pruning signal.* Here, 58 of 112 positions never modify the IR of any kernel, and 71.8% of prefixes yield a binary identical to their predecessor. An autotuner can hash IR or binaries to skip redundant measurements and to drop never-firing positions from a program's search space. A position inert in the canonical order may still enable later passes or matter in a reordered pipeline, so pruning decisions must be revalidated end-to-end.
4) *The canonicalization core is load-bearing in both directions.* The unsupervised ranking recovers the hand-tuned core (`early-cse`, `licm`, `instcombine`, `loop-vectorize`, `loop-rotate`, ...), and the same core causes most of the persistent loss. Tuning its heuristics (e.g., the iteration limit of the first `instcombine`) is therefore more promising than pruning the long tail.

Which Programs Benefit. Pass effects are strongly program-specific. Universal passes (`early-cse`, `sroa`, `loop-rotate`, `loop-unroll`: 23–24 kernels with a significant change) coexist with selective ones (`gvn`: 7 kernels). Dense linear algebra and stencils profit most from `loop-vectorize` and `loop-unroll`, the triangular solvers pay for the same passes in LLC misses, and the `datamining` reductions end near $1.0\times$ because mid-pipeline gains are undone later. A per-program pipeline should therefore key on loop structure and working-set size, which are cheap static features, not on IR size, which RQ_2 shows to be uninformative.

Implications for Cost Models and MLIR. IR-instruction count and runtime are essentially uncorrelated both across and within kernels (Fig. 5). Any cost model that uses IR count as a dominant runtime proxy will misrank pipeline variants and must be augmented with dynamic or microarchitectural signals (cycles, IPC, D1 misses) [40], [56]. Conversely, counters such as retired instructions and cycles are near-deterministic, so Bayesian-optimization priors built over them can treat per-pass effects as essentially deterministic. For compute-bound kernels, passes that reduce runtime also reduce energy by 30–60% (Fig. 7), so `-O3` is a free energy intervention for this class. Transfer to memory-bound, batched, or accelerator-hosted ML requires validation (§VI). On the dense solvers, late `loop-vectorize` and `loop-unroll` expand the data working set and inflate LLC misses. An MLIR lowering pipeline could gate these passes behind a working-set/LLC-capacity predicate.

Methodology Recommendations. Pass-level studies should calibrate significance on a null. In our data, $p < 0.05$ alone flags 25.7% of transitions whose binaries are byte-identical, so naive per-pass testing would report hundreds of spurious effects. Combining an effect-size floor with a test, and checking binary identity, reduces this to 0.9%. Reporting mean and median jointly, and absolute and relative impact jointly, caught estimator- and scale-driven artifacts at negligible cost. All recommendations are conditioned on PolyBench (default dataset), LLVM 21.1.8, `-O3`, single-threaded execution, and

Intel Alder Lake. Re-validation on SPEC CPU, `-Os`/`-Oz`, and other platforms is required before generalizing.

VI. Threats to Validity

The discussion follows the taxonomy of Wohlin *et al.* [82].

Conclusion Validity. We keep the 95% confidence interval under 1% of the mean for every metric and report median and mean jointly [79], [80], so estimator-driven artifacts surface instead of hiding. Every significance claim uses a criterion calibrated on 2,434 byte-identical transitions, with an empirical false-positive rate of 0.9%. Benjamini–Hochberg correction across the 113×30 grid changes the counts by at most one. We dropped Cohen's d because vanishing variance on near-deterministic transitions inflates it without bound. The decomposition of L depends on the recovery window. With one pass instead of three, mean persistent loss is 24.2 points, and with ten it is 16.4, so the qualitative conclusion (most of L is not persistent) does not hinge on this choice. Finally, L is an upper bound on forfeited speedup, not an estimate of what is recoverable.

Internal Validity. The differential design ascribes the $B_{i-1} \to B_i$ delta to p_i, but this delta is context- and order-dependent: a pass inert in the canonical `-O3` order may be load-bearing for later passes or in a reordered pipeline. We thus refrain from inferring global dispensability and require end-to-end revalidation before pruning (§V). System confounds are controlled as in §III (core pinning, RT-FIFO scheduling, `performance` governor, disabled ASLR), with cache references, RSS, page faults, and context switches as checks. Runtime and counters are collected separately to avoid instrumentation overhead. The residual drift visible on byte-identical binaries is absorbed by the calibrated criterion.

Construct Validity. "Marginal pass impact" is operationalized as the in-context prefix delta, which measures contribution *within* the canonical order. This matches how `-O3` deploys passes but is not a standalone-effect measurement. The pass sequence is extracted statically (`opt -print-pipeline-passes`) and flattened. Each prefix keeps the pass-manager nesting, so a function pass runs once per function and loop passes run per loop, exactly as in `-O3`. On-demand analyses, however, are not enumerated, and a pass that is invoked on several functions counts as a single transition. We complement the static view with the dynamic evidence that matters for attribution, namely whether the IR and binary actually change at each transition (§IV-A). Runtime, `clang+opt+llc` compile time, and binary size are direct measures. RAPL package energy excludes the DRAM and GPU domains, under-capturing full-system energy for memory-bound workloads.

External Validity. The findings are conditioned on PolyBench/C 4.2.1 (default dataset), LLVM 21.1.8, `-O3`, and single-threaded execution on one Intel Alder Lake host. PolyBench's regular affine loop nests isolate single-pass effects but do not represent irregular, pointer-heavy, or multi-threaded code. Its default working sets fit in the LLC, so memory-bound behavior is under-represented. Larger suites would mask single-pass effects or exceed our repetition budget (§III), so cross-suite, larger-dataset, and cross-architecture replication is required before generalizing. We expect structural findings (concentration in the canonicalization core, non-monotonicity, backloading) to transfer best, because they reflect the design of the pipeline, while energy and LLC/working-set effects depend on the host's RAPL domain and cache hierarchy. The ranking is version-locked. The MLIR transfer argument rests on structural similarity (§III) and is an unvalidated hypothesis.

VII. Related Work

We are not aware of any systematic study quantifying individual LLVM pass impact across multiple performance dimensions. We review the relevant strands below.

Search-Based and Learning-Based Optimization. Iterative compilation [26], [27], [83] explores the optimization space by evaluating compiler configurations: small search budgets can outperform `-O3` [26], [49] with program-dependent optimal configurations [45], [47], [84], while heuristic [42]–[44] and ML [36], [46], [85] approaches, collaborative filtering [86], and graph-based models [35] address the combinatorial space. Autotuning [40] adds Bayesian optimization [87], neural predictors [32], [46], and reinforcement learning (AutoPhase [81], CompilerGym [88]), with recent work reducing search cost via multi-phase learning [39], sub-sequence modeling [38], and proxy metrics [31]; a recurring limitation is reliance on expensive or inaccurate performance signals. More broadly, ML links hardware counters to optimization decisions [56] or uses deep learning for vectorization [89] and phase ordering [81], yet surveys [40] highlight the lack of fine-grained training data and difficulty isolating pass-level effects. Our work is orthogonal and complementary: we quantify each pass's marginal contribution across runtime, binary size, and counters, producing fine-grained data for supervision, surrogate targets, or priors while filling this pass-level data gap.

Compiler Fuzzing and Testing. Csmith [90], EMI [91], YARPGen [92], and CLsmith [93] target correctness, not performance. Our work is complementary: correctness enables reliable measurement, while performance characterization can guide fuzzing toward less understood optimizations.

Closest Work. Three lines of work are closest to ours, and they differ in objective, method, and findings. De La Torre *et al.* [57] share our objective of attributing performance to LLVM passes, but they study a limited set of passes and execution time only. We cover the complete `-O3` pipeline in its deployed order, add four further dimensions, and calibrate significance against a null, which lets us separate real regressions from noise. Chen *et al.* [58] deconstruct iterative optimization in GCC/ICC and find that a few optimizations carry most of the benefit. Our concentration result agrees, on a different compiler family, under scale-free measures, and on four further dimensions, which suggests that skewed pass impact is a property of optimization pipelines and not of a specific compiler. Phase-ordering studies [49], [59], [94] measure what a better order could gain, relative to orders found by search. We instead ask how much the *default* order forfeits along its own trajectory, which needs no search, and find that most of the apparent loss is noise or transient.

Performance Analysis of Compiler Optimizations. Cavazos *et al.* [56] use counters to predict optimization benefits, De-

mertzi *et al.* [55] analyze reliability, Popescu and Lopes [54] study undefined behavior, and Tan *et al.* [53] identify missed optimizations. None provides a cumulative, multi-dimensional evaluation of the full LLVM `-O3` pipeline.

Superoptimization and PGO. Our ranking, dominated by `instcombine` and `early-cse`, corroborates superoptimization results [95], [96]: peephole algebraic simplifications account for a disproportionate share of achievable optimizations. Our finding that pass-budget decisions can be made per pipeline concerns *compile-time allocation*, whereas PGO/FDO [97] shows that *which transformations to apply* benefits from per-program profiles. The two are compatible.

VIII. Conclusion

We presented a systematic, multi-dimensional study of the LLVM `-O3` pipeline, decomposing it into cumulative per-pass prefixes across $84{,}750$ measurements over 113 variants of 30 PolyBench/C kernels, and calibrating every significance claim on the transitions that leave the binary unchanged. The pipeline is non-monotone (36% of significant runtime changes are regressions) and back-loaded, and a small core of passes drives the gains under both absolute and scale-free measures. The final `-O3` binary is only marginally Pareto-dominated, except on four kernels where a strictly smaller and faster prefix exists. IR-instruction count is an unreliable runtime proxy, and runtime-targeted passes are *de facto* energy-targeted (30–60%). The search-free upper bound on speedup forfeited by non-monotonicity is $L = 46.35\%$, but only 21.3 points of it persist to the end of the pipeline, mostly caused by the same canonicalization passes that deliver the speedup. These results point pass pruning, phase-ordering repair, and cost-model calibration to a handful of pass positions. Future work targets SPEC CPU, larger datasets, `-Os`/`-Oz`, more platforms, and MLIR-lowered ML kernels.

References

[1] A. P. Ershov, "On Programming of Arithmetic Operations," *Communications of the ACM*, vol. 1, no. 8, pp. 3–6, Aug. 1958.

[2] J. Heller, "Sequencing Aspects of Multiprogramming," *Journal of ACM*, vol. 8, no. 3, pp. 426–439, Jul. 1961.

[3] W. M. McKeeman, "Peephole Optimization," *Communications of the ACM*, vol. 8, no. 7, pp. 443–444, Jul. 1965.

[4] C. W. Gear, "High Speed Compilation of Efficient Object Code," *Communications of the ACM*, vol. 8, no. 8, pp. 483–488, Aug. 1965.

[5] E. S. Lowry and C. W. Medlock, "Object Code Optimization," *Communications of the ACM*, vol. 12, no. 1, pp. 13–22, Jan. 1969.

[6] V. A. Busam and D. E. Englund, "Optimization of Expressions in Fortran," *Commu. of the ACM*, vol. 12, no. 12, pp. 666–674, Dec. 1969.

[7] F. E. Allen, "Program Optimization," in *Annual Review of Automatic Programming*. Pergamon Press, 1969, vol. 5, pp. 239–307.

[8] W. A. Wulf, *The Design of an Optimizing Compiler*. CMU, Dec. 1973.

[9] K. D. Cooper and L. Torczon, *Engineering a Compiler*, Nov. 2022.

[10] K. Kennedy and J. R. Allen, *Optimizing Compilers for Modern Architectures: A Dependence-Based Approach*. Morgan Kaufmann, Oct. 2001.

[11] R. Paige, "Future Directions in Program Transformations," *ACM Computing Surveys*, vol. 28, no. 4, pp. 170–174, Dec. 1996.

[12] D. F. Bacon, S. L. Graham, and O. J. Sharp, "Compiler Transformations for High-Performance Computing," *ACM Computing Surveys*, vol. 26, no. 4, pp. 345–420, Dec. 1994.

[13] M. Dodds, M. Batty, and A. Gotsman, "Compositional Verification of Compiler Optimisations on Relaxed Memory," in *Proc. ESOP'18*, LNCS 10801, Thessaloniki, Greece: Springer, Apr. 2018, pp. 1027–1055.

[14] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.

[15] C. Lattner and V. Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation," in *Proc. CGO'04*, San José, CA, USA: IEEE, Mar. 2004, pp. 75–86.

[16] N. D. Matsakis and F. S. Klock, "The Rust Language," *ACM SIGAda Letters*, vol. 34, no. 3, pp. 103–104, Oct. 2014.

[17] J. Bezanson, S. Karpinski, V. B. Shah, and A. Edelman, "Julia: A Fast Dynamic Language for Technical Computing," *arXiv e-prints*, arXiv:1209.5145, pp. 1–27, Sep. 2012.

[18] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling Compiler Infrastructure for Domain Specific Computation," in *Proc. CGO'21*. Virtual: IEEE, Feb./Mar. 2021, pp. 2–14.

[19] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Proc. NeurIPS'19*. Vancouver, Canada: Curran Associates, Inc., Dec. 2019, pp. 8026–8037.

[20] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: A System for Large-Scale Machine Learning," in *Proc. OSDI'16*. Savannah, GA, USA: USENIX, Nov. 2016, pp. 265–283.

[21] M. Li, Y. Liu, X. Liu, Q. Sun, X. You, H. Yang, Z. Luan, L. Gan, G. Yang, and D. Qian, "The Deep Learning Compiler: A Comprehensive Survey," *IEEE Trans. Par. and Distr. Syst.*, vol. 32, no. 3, pp. 708–727, Mar. 2021.

[22] Y. Xing, J. Weng, Y. Wang, L. Sui, Y. Shan, and Y. Wang, "An In-Depth Comparison of Compilers for Deep Neural Networks on Hardware," in *Proc. ICESS'19*. Las Vegas, NV, USA: IEEE, Jun. 2019, pp. 1–8.

[23] R. Zhao, S. Liu, H.-C. Ng, E. Wang, J. J. Davis, X. Niu, X. Wang, H. Shi, G. A. Constantinides, P. Y. K. Cheung, and W. Luk, "Hardware Compilation of Deep Neural Networks: An Overview," in *Proc. ASAP'18*. Milan, Italy: IEEE, Jul. 2018, pp. 1–8.

[24] H.-I. C. Liu, M. Brehler, M. Ravishankar, N. Vasilache, B. Vanik, and S. Laurenzo, "TinyIREE: An ML Execution Environment for Embedded Systems from Compilation to Deployment," *IEEE Micro*, vol. 42, no. 5, pp. 9–16, Sep. 2022.

[25] T. Chen, T. Moreau, Z. Jiang, H. Shen, E. Q. Yan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," *arXiv e-prints*, arXiv:1802.04799, pp. 1–16, Oct. 2018.

[26] K. D. Cooper, P. J. Schielke, and D. Subramanian, "Optimizing for Reduced Code Space Using Genetic Algorithms," in *Proc. LCTES'99*, Atlanta, GA, USA: ACM, May 1999, pp. 1–9.

[27] K. D. Cooper, D. Subramanian, and L. Torczon, "Adaptive Optimizing Compilers for the 21st Century," *J. Supercomputing*, vol. 23, no. 1, pp. 7–22, Aug. 2002.

[28] S. Triantafyllis, M. Vachharajani, N. Vachharajani, and D. I. August, "Compiler Optimization-Space Exploration," in *Proc. CGO'03*, San Francisco, CA, USA: IEEE, Mar. 2003, pp. 204–215.

[29] S.-A.-A. Touati and D. Barthou, "On the Decidability of Phase Ordering Problem in Optimizing Compilation," in *Proc. CF'06*, Ischia, Italy: ACM, May 2006, pp. 147–156.

[30] S. M. Freudenberger and J. C. Ruttenberg, "Phase Ordering of Register Allocation and Instruction Scheduling," in *Proc. WoCC'92*. Daghstul, Germany: Springer, May 1991, pp. 146–170.

[31] J. Zhao, C. Xia, and Z. Wang, "Leveraging Compilation Statistics for Compiler Phase Ordering," in *Proc. IPDPS'25*. Milan, Italy: IEEE, Jun. 2025, pp. 533–545.

[32] A. H. Ashouri, A. Bignoli, G. Palermo, C. Silvano, S. Kulkarni, and J. Cavazos, "MiCOMP: Mitigating the Compiler Phase-Ordering Problem Using Optimization Sub-Sequences and Machine Learning," *ACM Trans. Arch. Code Opt.*, vol. 14, no. 3, pp. 29:1–29:28, Sep. 2017.

[33] J. W. Davidson, G. S. Tyson, D. B. Whalley, and P. A. Kulkarni, "Evaluating Heuristic Optimization Phase Order Search Algorithms," in *Proc. CGO'07*. San José, CA, USA: IEEE, Mar. 2007, pp. 157–169.

[34] S. R. Vegdahl, "Phase Coupling and Constant Generation in an Optimizing Microcode Compiler," in *Proc. MICRO'82*, Oct. 1982, pp. 125–133.

[35] R. Nobre, L. G. A. Martins, and J. a. M. P. Cardoso, "A Graph-Based Iterative Compiler Pass Selection and Phase Ordering Approach," in *Proc. LCTES'16*. Santa Barbara, USA: ACM, Jun. 2016, pp. 21–30.

[36] F. Agakov, E. V. Bonilla, J. Cavazos, B. Franke, G. Fursin, M. F. P. O'Boyle, J. Thomson, M. Toussaint, and C. K. I. Williams, "Using Machine Learning to Focus Iterative Optimization," in *Proc. CGO'06*, New York, NY, USA: IEEE, Mar. 2006, pp. 305–315.

[37] D. L. Whitfield and M. L. Soffa, "An Approach to Ordering Optimizing Transformations," in *Proc. PPoPP'90*, ACM, Mar. 1990, pp. 137–146.

[38] H. Pan, Y. Wei, M. Xing, Y. Wu, and C. Zhao, "Towards Efficient Compiler Auto-tuning: Leveraging Synergistic Search Spaces," in *Proc. CGO'25*. Las VEgas, NV, USA: IEEE, Mar. 2025, pp. 614–627.

[39] M. Zhu, D. Hao, and J. Chen, "Compiler Autotuning through Multiple-phase Learning," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 4, pp. 1–38, May 2024.

[40] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, "A Survey on Compiler Autotuning using Machine Learning," *ACM Computing Surveys*, vol. 51, no. 5, pp. 96:1–96:42, Sep. 2018.

[41] A. Tiwari, C. Chen, J. Chame, M. Hall, and J. K. Hollingsworth, "A Scalable Auto-Tuning Framework for Compiler Optimization," in *Proc. IPDPS'09*, Rome, Italy: IEEE, May 2009, pp. 1–12.

[42] Z. Pan and R. Eigenmann, "PEAK—A Fast and Effective Performance Tuning System via Compiler Optimization Orchestration," *ACM Transactions on Programming Languages and Systems*, vol. 30, no. 3, pp. 17:1–17:43, May 2008.

[43] ——, "Fast and Effective Orchestration of Compiler Optimizations for Automatic Performance Tuning," in *Proc. CGO'06*, 2006, pp. 319–332.

[44] P. Kulkarni, S. Hines, J. Hiser, D. Whalley, J. Davidson, and D. Jones, "Fast Searches for Effective Optimization Phase Sequences," in *Proc. PLDI'04*, Washington, DC, USA: ACM, Jun. 2004, pp. 171–182.

[45] S. Purini and L. Jain, "Finding Good Optimization Sequences Covering Program Space," *ACM Trans. Arch. Code Opt.*, vol. 9, no. 4, Jan. 2013.

[46] S. Kulkarni and J. Cavazos, "Mitigating the Compiler Optimization Phase-Ordering Problem Using Machine Learning," in *Proc. OOPSLA'12*. Tucson, AZ, USA: ACM, Oct. 2012, pp. 147–162.

[47] P. Kulkarni, W. Zhao, H. Moon, K. Cho, D. Whalley, J. Davidson, M. Bailey, Y. Paek, and K. Gallivan, "Finding Effective Optimization Phase Sequences," in *Proc. LCTES'03*, ACM, Jun. 2003, pp. 12–23.

[48] D. L. Whitfield and M. L. Soffa, "An Approach for Exploring Code Improving Transformations," *ACM Transactions on Programming Languages and Systems*, vol. 19, no. 6, pp. 1053–1084, Nov. 1997.

[49] L. Almagor, K. Cooper, A. Grosul, T. Harvey, S. Reeves, D. Subramanian, L. Torczon, and T. Waterman, "Finding Effective Compilation Sequences," in *Proc. LCTES'04*, ACM, Jun. 2004, pp. 231–239.

[50] M. Zhao, B. R. Childers, and M. L. Soffa, "An Approach Toward Profit-Driven Optimization," *ACM Transactions on Architecture and Code Optimization*, vol. 3, no. 3, pp. 231–262, Sep. 2006.

[51] J. Liu, J. Fang, T. Wang, J. Xie, C. Huang, and Z. Wang, "Efficient Compiler Optimization by Modeling Passes Dependence," *CCF Transactions on High Performance Computing*, vol. 6, no. 6, pp. 588–607, Dec. 2024.

[52] K. Georgiou, C. Blackmore, S. Xavier-de Souza, and K. Eder, "Less is More: Exploiting the Standard Compiler Optimization Levels for Better Performance and Energy Consumption," in *Proc. SCOPES'18*, Sankt Goar, Germany: ACM, May 2018, pp. 35–42.

[53] J. Tan, S. Jiao, M. Chabbi, and X. Liu, "What Every Scientific Programmer Should Know about Compiler Optimizations?" in *Proc. ICS'20*. Barcelona, Spain: ACM, Jun./Jul. 2020, pp. 1–12.

[54] L. Popescu and N. P. Lopes, "Exploiting Undefined Behavior in C/C++ Programs for Optimization: A Study on the Performance Impact," in *Proc. PLDI'25*, Seoul, South Korea: ACM, Jun. 2025, pp. 348–371.

[55] M. Demertzi, M. Annavaram, and M. Hall, "Analyzing the Effects of Compiler Optimizations on Application Reliability," in *Proc. IISWC'11*, Austin, TX, USA: IEEE, Nov. 2011, pp. 184–193.

[56] J. Cavazos, G. Fursin, F. Agakov, E. Bonilla, M. F. P. O'Boyle, and O. Temam, "Rapidly Selecting Good Compiler Optimizations using Performance Counters," in *Proc. CGO'07*, Mar. 2007, pp. 185–197.

[57] J. C. de la Torre, P. Ruiz, B. Dorronsoro, and P. L. Galindo, "Analyzing the Influence of LLVM Code Optimization Passes on Software Performance," in *Proc. IPMU'18*, CCIS 855. Springer, Jun. 2018, pp. 272–283.

[58] Y. Chen, S. Fang, Y. Huang, L. Eeckhout, G. Fursin, O. Temam, and C. Wu, "Deconstructing Iterative Optimization," *ACM Trans. Arch. Code Opt.*, vol. 9, no. 3, pp. 21:1–21:30, Oct. 2012.

[59] P. A. Kulkarni, D. B. Whalley, G. S. Tyson, and J. W. Davidson, "Exhaustive Optimization Phase Order Space Exploration," in *Proc. CGO'06*, New York, NY, USA: IEEE, Mar. 2006, pp. 306–318.

[60] N. van Kempen, H.-J. Kwon, D. T. Nguyen, and E. D. Berger, "It's Not Easy Being Green: On the Energy Efficiency of Programming Languages," in *Proc. ASE'25*, IEEE, Nov. 2025, pp. 1553–1565.

[61] S. U. Lee, N. Fernando, K. Lee, and J.-G. Schneider, "A Survey of Energy Concerns for Software Engineering," *J. Syst. Soft.*, vol. 210, Apr. 2024.

[62] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, "Energy Efficiency Across Programming Languages," in *Proc. SLE'17*. Vancouver, BC, Canada: ACM, Oct. 2017, pp. 256–267.

[63] B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Global Value Numbers and Redundant Computations," in *Proc. PoPL'88*, Jan. 1988, pp. 12–27.

[64] B. Alpern, M. N. Wegman, and F. K. Zadeck, "Detecting Equality of Variables in Programs," in *Proc. PoPL'88*, ACM, Jan. 1988, pp. 1–11.

[65] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "An Efficient Method of Computing Static Single Assignment Form," in *Proc. POPL'89*, Austin, TX, USA: ACM, Jan. 1989, pp. 25–35.

[66] D. Callahan, K. D. Cooper, K. Kennedy, and L. Torczon, "Interprocedural Constant Propagation," in *Proc. SCC'86*, ACM, Jun. 1986, pp. 152–161.

[67] M. N. Wegman and F. K. Zadeck, "Constant Propagation with Conditional Branches," *ACM Transactions on Programming Languages and Systems*, vol. 13, no. 2, pp. 181–210, Apr. 1991.

[68] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently Computing Static Single Assignment Form and the Control Dependence Graph," *ACM Transactions on Programming Languages and Systems*, vol. 13, no. 4, pp. 451–490, Oct. 1991.

[69] S. S. Muchnick, *Advanced Compiler Design and Implementation*, 1st ed. Morgan Kaufmann, Aug. 1997.

[70] S. Carr and K. Kennedy, "Scalar Replacement in the Presence of Conditional Control Flow," *Software—Practice and Experience*, vol. 24, no. 1, pp. 51–77, Jan. 1994.

[71] J. Lu and K. D. Cooper, "Register Promotion in C Programs," in *Proc. PLDI'97*, Las Vegas, NV, USA: ACM, Jun. 1997, pp. 308–319.

[72] P. P. Chang and W.-W. Hwu, "Inline Function Expansion for Compiling C Programs," in *Proc. PLDI'89*, Portland, USA: Jun. 1989, pp. 246–257.

[73] J. J. Dongarra and A. R. Hinds, "Unrolling loops in Fortran," *Software—Practice and Experience*, vol. 9, no. 3, pp. 219–226, Mar. 1979.

[74] J. W. Davidson and S. Jinturkar, "An Aggressive Approach to Loop Unrolling," University of Virginia, VA, USA, TR-CS-95-26, Jan. 1995.

[75] J. Knoop, O. Rüthing, and B. Steffen, "Partial Dead Code Elimination," in *Proc. PLDI'94*, Orlando, FL, USA: ACM, Jun. 1994, pp. 147–158.

[76] F. E. Allen and J. Cocke, "A Catalogue of Optimizing Transformations," in *Design and Optimization of Compilers*, Prentice-Hall, 1972, pp. 1–30.

[77] T. Yuki, "Understanding PolyBench/C 3.2 Kernels," in *Proc. IMPACT'14*, Wien, Austria, Jan. 2014, pp. 1–5.

[78] V. M. Weaver and S. A. McKee, "Can Hardware Performance Counters Be Trusted?" in *Proc. IISWC'08*, IEEE, Sep. 2008, pp. 141–150.

[79] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing Wrong Data Without Doing Anything Obviously Wrong!" in *Proc. ASPLOS'09*, Washington, USA: ACM, Mar. 2009, pp. 265–276.

[80] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically Rigorous Java Performance Evaluation," in *Proc. OOPSLA'07*, Montréal, Canada: ACM, Oct. 2007, pp. 57–76.

[81] A. Haj-Ali, Q. Huang, J. Xiang, W. Moses, K. Asanovic, J. Wawrzynek, and I. Stoica, "AutoPhase: Juggling HLS Phase Orderings in Random Forests with Deep Reinforcement Learning," in *Proc. MLSys'20*, Austin, TX, USA: Curran, Mar. 2020, pp. 70–81.

[82] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.

[83] F. Bodin, T. Kisuki, P. Knijnenburg, M. O'Boyle, and E. Rohou, "Iterative Compilation in a Non-linear Optimisation Space," in *Proc. PFDC'98*, Paris, France, Oct. 1998.

[84] L. Eeckhout, H. Vandierendonck, and K. De Bosschere, "Quantifying the Impact of Input Data Sets on Program Behavior and Its Applications," *Journal of Instruction-Level Parallelism*, vol. 5, no. 1, pp. 1–33, Feb. 2003.

[85] G. Fursin, Y. Kashnikov, A. W. Memon, Z. Chamski, O. Temam, M. Namolaru, E. Yom-Tov, B. Mendelson, A. Zaks, E. Courtois, F. Bodin, P. Barnard, E. Ashton, E. Bonilla, J. Thomson, C. K. I. Williams, and M. O'Boyle, "Milepost GCC: Machine Learning Enabled Self-Tuning Compiler," *International Journal of Parallel Programming*, vol. 39, no. 3, pp. 296–327, Jan. 2011.

[86] H. Liu, J. Luo, Y. Li, and Z. Wu, "Iterative Compilation Optimization Based on Metric Learning and Collaborative Filtering," *ACM Transactions on Architecture and Code Optimization*, vol. 19, no. 1, pp. 2:1–2:25, Mar. 2021.

[87] A. H. Ashouri, G. Mariani, G. Palermo, E. Park, J. Cavazos, and C. Silvano, "COBAYN: Compiler Autotuning Framework Using Bayesian Networks," *ACM Transactions on Architecture and Code Optimization*, vol. 13, no. 2, pp. 21:1–21:25, Jun. 2016.

[88] C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather, "CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research," in *Proc. CGO'22*, Seoul, South Korea: IEEE, Apr. 2022, pp. 92–105.

[89] A. Haj-Ali, N. K. Ahmed, T. Willke, Y. S. Shao, K. Asanovic, and I. Stoica, "NeuroVectorizer: End-to-End Vectorization with Deep Rein-

forcement Learning," in *Proc. CGO'20*, San Diego, CA, USA: IEEE, Feb. 2020, pp. 242–255.
[90] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and Understanding Bugs in C Compilers," in *Proc. PLDI'11*, San Josè, CA, USA: ACM, Jun. 2011, pp. 283–294.
[91] V. Le, M. Afshari, and Z. Su, "Compiler Validation via Equivalence Modulo Inputs," in *Proc. PLDI'14*, Edinburgh, United Kingdom: ACM, Jun. 2014, pp. 216–226.
[92] V. Livinskii, D. Babokin, and J. Regehr, "Random Testing for C and C++ Compilers with YARPGen," in *Proc. OOPSLA'20*, Chicago, IL, USA: ACM, Nov. 2020, pp. 1–25.
[93] C. Lidbury, A. Lascu, N. Chong, and A. F. Donaldson, "Many-Core Compiler Fuzzing," in *Proc. PLDI'15*, Portland, OR, USA: ACM, Jun. 2015, pp. 65–76.
[94] M. R. Jantz and P. A. Kulkarni, "Analyzing and Addressing False Interactions During Compiler Optimization Phase Ordering," *Software—Practice and Experience*, vol. 44, no. 6, pp. 643–679, Jun. 2014.
[95] H. Massalin, "Superoptimizer: A Look at the Smallest Program," in *Proc. ASPLOS'87*, Palo Alto, CA, USA: ACM, Oct. 1987, pp. 122–126.
[96] S. Bansal and A. Aiken, "Automatic Generation of Peephole Superoptimizers," in *Proc. ASPLOS'06*, ACM, Oct. 2006, pp. 394–403.
[97] D. Chen, T. Moseley, and D. X. Li, "AutoFDO: Automatic Feedback-Directed Optimization for Warehouse-Scale Applications," in *Proc. CGO'16*, Barcelona, Spain: IEEE, Mar. 2016, pp. 12–23.